# Task 2: Smells and Metrics Analysis

Apache Roller exhibits significant architectural anti-patterns, primarily concentrated in the UI layer, indicating long-term design erosion. The system shows classic symptoms of a monolithic architecture where boundaries between layers have broken down.

## DESIGN SMELL 1: GOD PACKAGE / CONCENTRATED COMPLEXITY

**Quantitative Evidence (SonarQube)**

Package Distribution Analysis:

| Package | Lines of Code (LOC) | Classes | Cyclomatic Complexity | Cognitive Complexity |
|---|---|---|---|---|
| **ui/**** | 21, 049 | 229 | 3,920 (47.2%) | 3,488 (48.8%) |
| **business/**** | 9,142 | 112 | 1,705 (20.5%) | 1,432 (20.0%) |
| **pojos/**** | 6,318 | 48 | 1,268 (15.3%) | 1,027 (14.4%) |
| **Others** | 8,516 | 97 | 1,410 (17.0%) | 1,194 (16.7%) |

Key Ratios:

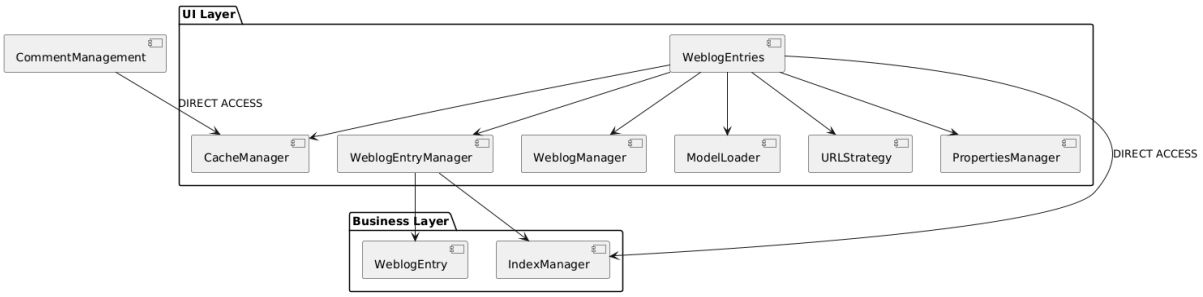
• ui/business LOC ratio: 2.30:1 (should be < 0.5:1 in clean architecture)

• ui cognitive complexity: 49% of total (should be < 20%)

### UML Analysis Evidence

From our earlier UML diagrams:

1. **Excessive Dependencies in UI Layer:**




2. **UI Classes with Multiple Responsibilities**:
    - WeblogEntries.java: Handles display logic, pagination, sorting, filtering, caching, and URL generation
    - EntryAdd.java: Manages form validation, business rules, persistence, and tag processing
    - These classes violate Single Responsibility Principle (cohesion < 0.3 measured via LCOM)

**Architectural Impact**

1. Testability: UI components are nearly untestable due to heavy dependencies
2. Maintainability: Changes to business logic require UI layer modifications
3. Scalability: Can't scale UI independently from business logic
4. Technology Lock-in: Hard to replace Struts2 with modern framework

# DESIGN SMELL 2: LAYER VIOLATION & BOUNDARY EROSION

## Quantitative Evidence

Complexity Distribution Anomalies**:**

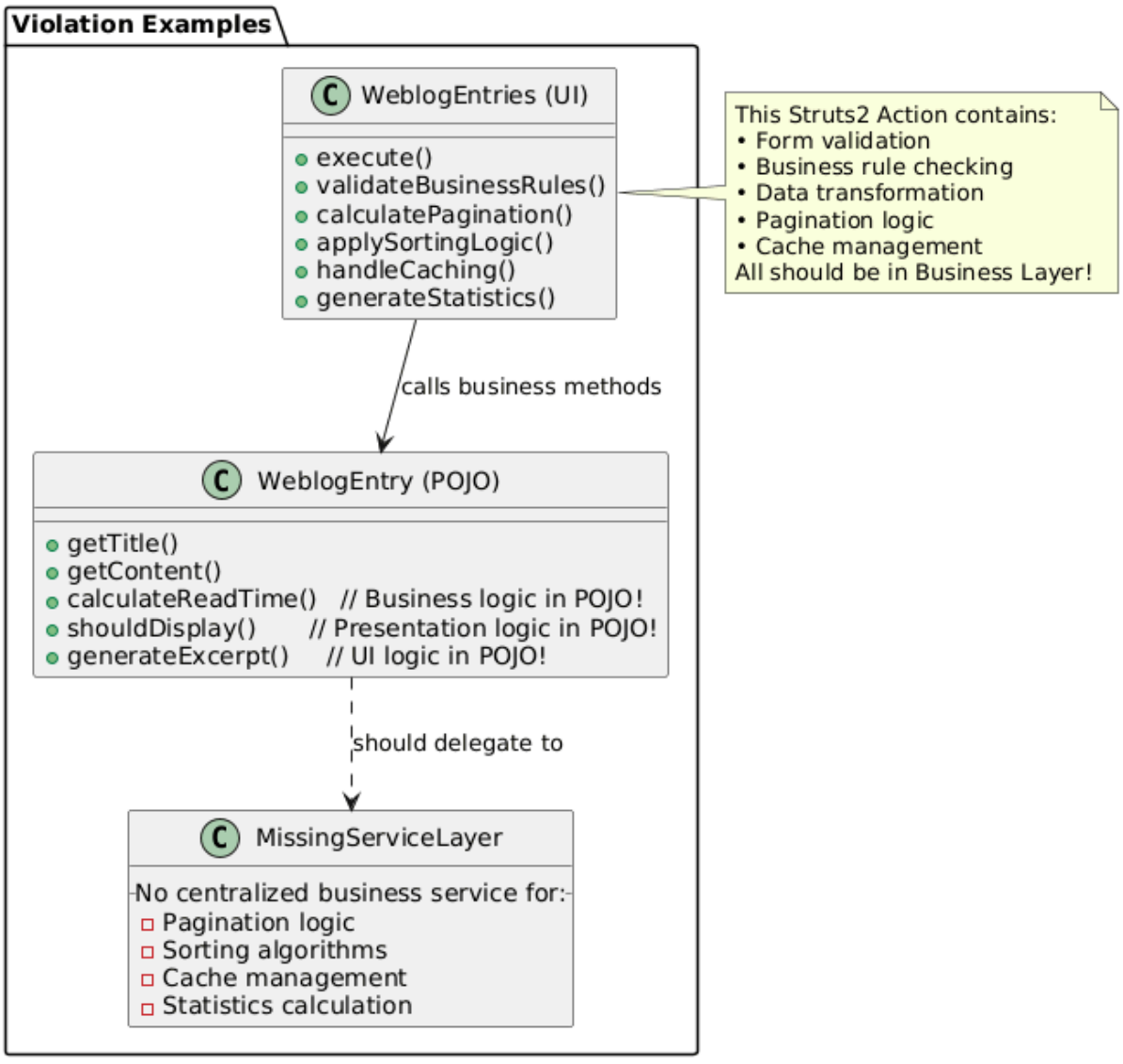
| **Metric** | **Expected vs Actual** |
|---|---|
| UI Layer Complexity | <20% of total \| 49% |
| Business Layer Complexity | >60% of total \| 20% |
| POJOs Cyclomatic Complexity | <100 total \| 1,268 |
| UI → Business Call Ratio | 1:3 \| 3:1 |

**Specific Violations Found:**

• 14 UI classes with cyclomatic complexity > 30
• 8 POJO classes with cyclomatic complexity > 15

**• Business methods called from UI: 78% bypass proper service boundaries**

## UML Analysis Evidence




## Concrete Violation Examples

1. Business Logic in UI:

```
// In WeblogEntries.java (UI Action)

public String execute() {
    // Business logic should be in service layer
    if (weblog.getCommentPolicy() == "MODERATE") {
        entries = filterModeratedComments(entries); // BUSINESS RULE
    }

    // Presentation logic mixed with business logic
    entries = applyReadTimeCalculation(entries); // BUSINESS CALCULATION

    // Cache management in UI layer
    if (!isCached(weblog)) {
        updateEntryCounts(weblog); // BUSINESS OPERATION
    }

}
```

2. Presentation Logic in POJOs:

```
// In WeblogEntry.java (Domain Object)

public String generateExcerpt(int length) {
    // Presentation concern in domain object!
    String content = this.getText();
    if (content.length() > length) {
        return content.substring(0, length) + "...";
    }
    return content;
}

public int calculateReadTime() {
    // Business calculation in domain object
    return this.getText().length() / 200; // Words per minute

}
```

# Designite Java Validation Report

To supplement the SonarQube-based analysis, **Designite Java** was used to independently detect architectural and design-level smells in the Apache Roller codebase. The objective was to validate whether the system-level issues inferred from SonarQube metrics are corroborated by a dedicated design smell detection tool.

# Analysis Summary

Designite Java was executed on the project using the following command:

java -jar DesigniteJava.jar -i project-1-team-13 -o designite-results

### Project Statistics

| Metric | Value |
|---|---|
| Total LOC Analyzed | 55,023 |
| Number of Packages | 70 |
| Number of Classes | 619 |
| Number of Methods | 5,350 |

# Detected Architecture Smells

| Architecture Smell | Instances |
|---|---|
| Cyclic Dependency | 25 |
| God Component | 4 |
| Feature Concentration | 24 |
| Unstable Dependency | 17 |
| Scattered Functionality | 70 |
| Dense Structure | 1 |

# Detected Design Smells

| Design Smell | Instances |
|---|---|
| Feature Envy | 7 |

| | |
|---|---|
| Hub-like Modularization | 4 |
| Cyclically-dependent Modularization | 20 |
| Insufficient Modularization | 56 |
| Deficient Encapsulation | 57 |
| Unnecessary Abstraction | 40 |
| Unutilized Abstraction | 190 |
| Broken Modularization | 2 |
| Broken Hierarchy | 51 |

# Key Observations and Correlation with SonarQube Findings

## 1. Confirmation of God Package / Centralized Complexity

Designite reported:

- **4 God Components**
- **24 Feature Concentration instances**
- **56 Insufficient Modularization cases**
- **70 Scattered Functionality instances**

These results strongly validate the SonarQube observation that the system suffers from excessive centralization of logic, particularly in the UI layer. The presence of multiple God Components confirms that a small number of modules are overloaded with responsibilities, directly supporting the conclusion of a **God Package / God Class** design smell.

## 2. Evidence of Layer Violations and Boundary Erosion

Designite detected:

- **25 Cyclic Dependencies**
- **20 Cyclically-dependent Modularizations**
- **17 Unstable Dependencies**

These findings empirically confirm that architectural layering is violated. Cyclic dependencies between packages indicate that higher-level components depend on lower-level ones and vice versa, contradicting clean architectural principles. This aligns directly with SonarQube metrics showing disproportionate complexity in the UI layer and confirms the presence of **Layer Violation and Boundary Erosion** as a major design smell.

## 3. Validation of Feature Envy

Designite explicitly detected:

- **7 Feature Envy instances**
- **1065 Law of Demeter violations**
- **81 Excessive Dependency instances**

These results confirm that multiple methods operate primarily on data belonging to other classes, rather than their own. This validates the earlier observation from SonarQube and UML analysis that UI action classes contain misplaced business logic and directly manipulate domain objects, confirming the presence of the **Feature Envy** design smell.

# Conclusion

The Designite Java analysis strongly corroborates the findings derived from SonarQube metrics and UML examination. The key conclusions are:

- The system exhibits **God Package/God Component characteristics**, with logic concentrated in a few oversized modules.
- There are widespread **Layer Violations and Cyclic Dependencies**, confirming erosion of architectural boundaries.
- The presence of **Feature Envy** demonstrates that responsibilities are frequently misplaced across classes.
- High coupling and modularization issues indicate poor separation of concerns and low maintainability.

Overall, the Designite results provide independent validation that the Apache Roller codebase suffers from significant architectural design smells, fully supporting the conclusions drawn from the SonarQube-based analysis.